

CHƯƠNG 1

KIẾN TRÚC ỨNG DỤNG WEB HIỆN ĐẠI VÀ THIẾT KẾ RESTFUL API

Thời lượng: 4 LT | 4 TH — Công nghệ: .NET 10, MediatR, Mapster, Scalar

MỤC TIÊU CHƯƠNG

Sau khi hoàn thành chương này, sinh viên có khả năng:

Về kiến thức

- Phân tích và so sánh được các mô hình kiến trúc ứng dụng web: Monolithic, Layered, API-Driven và Vertical Slice Architecture; nhận biết ưu nhược điểm từng mô hình trong bối cảnh thực tế.
- Hiểu và giải thích được nguyên lý Clean Architecture, các tầng (Domain, Application, Infrastructure, Presentation) và quy tắc phụ thuộc theo chiều từ ngoài vào trong.
- Nắm vững mô hình CQRS (Command Query Responsibility Segregation) và cách MediatR triển khai pattern này trong .NET.
- Trình bày được các nguyên tắc thiết kế RESTful API: resource-oriented design, HTTP methods, status codes, idempotency, versioning và các kỹ thuật pagination/filtering/sorting.
- Hiểu vai trò của OpenAPI 3.x và cách Scalar tích hợp với .NET 10 để cung cấp tài liệu API tương tác.

Về kỹ năng

- Thiết lập được một solution .NET 10 theo chuẩn Clean Architecture với đầy đủ 4 tầng, cấu hình Dependency Injection và MediatR.
- Viết được các Command/Query Handler hoàn chỉnh sử dụng MediatR và Mapster cho DTO mapping.
- Thiết kế và triển khai RESTful API với .NET 10 Minimal APIs, hỗ trợ pagination, filtering, sorting với cấu trúc response chuẩn.
- Cấu hình và sử dụng Scalar để tài liệu hóa và kiểm thử API trong môi trường development.

Về thái độ

- Có ý thức tổ chức code theo kiến trúc phân tầng rõ ràng, tránh spaghetti code và god class.
- Tuân thủ nguyên tắc "design first" — thiết kế API contract trước khi cài đặt.

- Nhận thức được tầm quan trọng của khả năng bảo trì (maintainability) và khả năng kiểm thử (testability) khi lựa chọn kiến trúc.

GIỚI THIỆU CHƯƠNG

Trong phát triển phần mềm hiện đại, kiến trúc ứng dụng không còn là quyết định "một lần là xong" — nó quyết định trực tiếp đến khả năng mở rộng, bảo trì, kiểm thử và tốc độ phát triển của toàn bộ dự án. Chương này là nền tảng kỹ thuật cho toàn bộ giáo trình, thiết lập "ngôn ngữ kiến trúc" chung mà tất cả các chương sau đều xây dựng trên đó.

Chúng ta sẽ bắt đầu bằng hành trình tiến hóa của kiến trúc ứng dụng web — từ Monolithic đơn giản đến API-Driven Architecture hiện đại — để hiểu tại sao mỗi mô hình ra đời và khi nào nên áp dụng. Từ đó, ta đi sâu vào Clean Architecture, mô hình kiến trúc xuyên suốt giáo trình này, kết hợp với CQRS pattern và thư viện MediatR — bộ công cụ chuẩn trong hệ sinh thái .NET enterprise.

Phần trọng tâm tiếp theo là thiết kế RESTful API — bao gồm cả lý thuyết lẫn quyết định kỹ thuật thực tế như pagination, filtering, versioning. Cuối chương, sinh viên sẽ hoàn thành Case Study đầu tiên: khởi tạo nền tảng kỹ thuật cho dự án Culinary Blog (Blog ẩm thực) — dự án xuyên suốt toàn bộ giáo trình.

Lưu ý

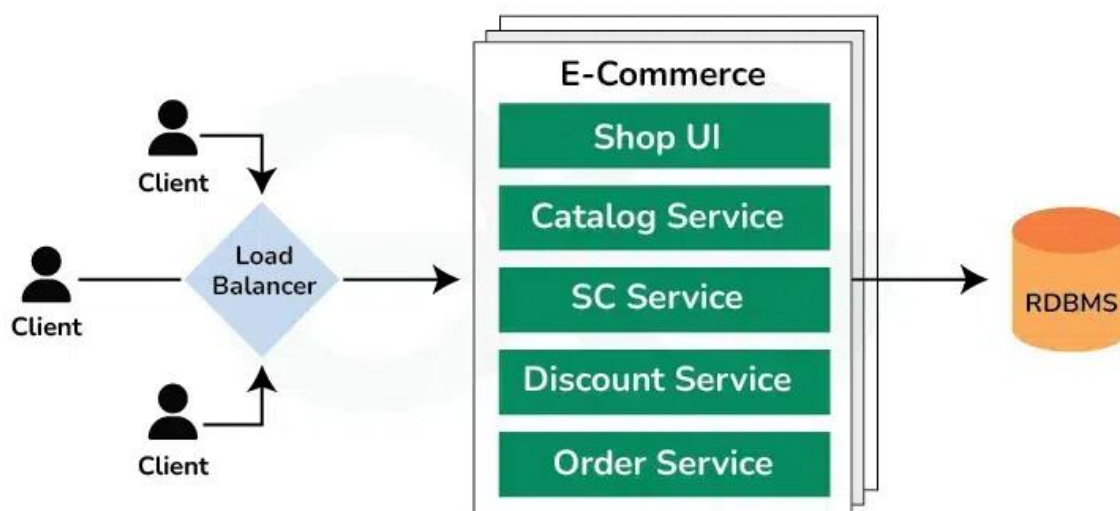
Dự án minh họa xuyên suốt giáo trình này là "Culinary Blog" — một ứng dụng web blog ẩm thực và công thức nấu ăn. Mỗi chương sẽ bổ sung một tầng kỹ thuật mới cho dự án này, từ API nền tảng (Chương 1) đến bảo mật (Chương 2), tối ưu hóa dữ liệu (Chương 3), giao diện người dùng (Chương 4) và cuối cùng là deployment production (Chương 6–7).

1.1. SỰ TIẾN HÓA CỦA KIẾN TRÚC ỨNG DỤNG WEB

1.1.1 Monolithic Architecture

Kiến trúc Monolithic (nguyên khối) là dạng kiến trúc phổ biến nhất trong giai đoạn đầu phát triển web. Toàn bộ ứng dụng — giao diện người dùng, logic nghiệp vụ, truy cập dữ liệu — được đóng gói trong một đơn vị triển khai duy nhất. Khi ứng dụng nhỏ, đây là lựa chọn đơn giản và hiệu quả: dễ phát triển, dễ debug, dễ triển khai.

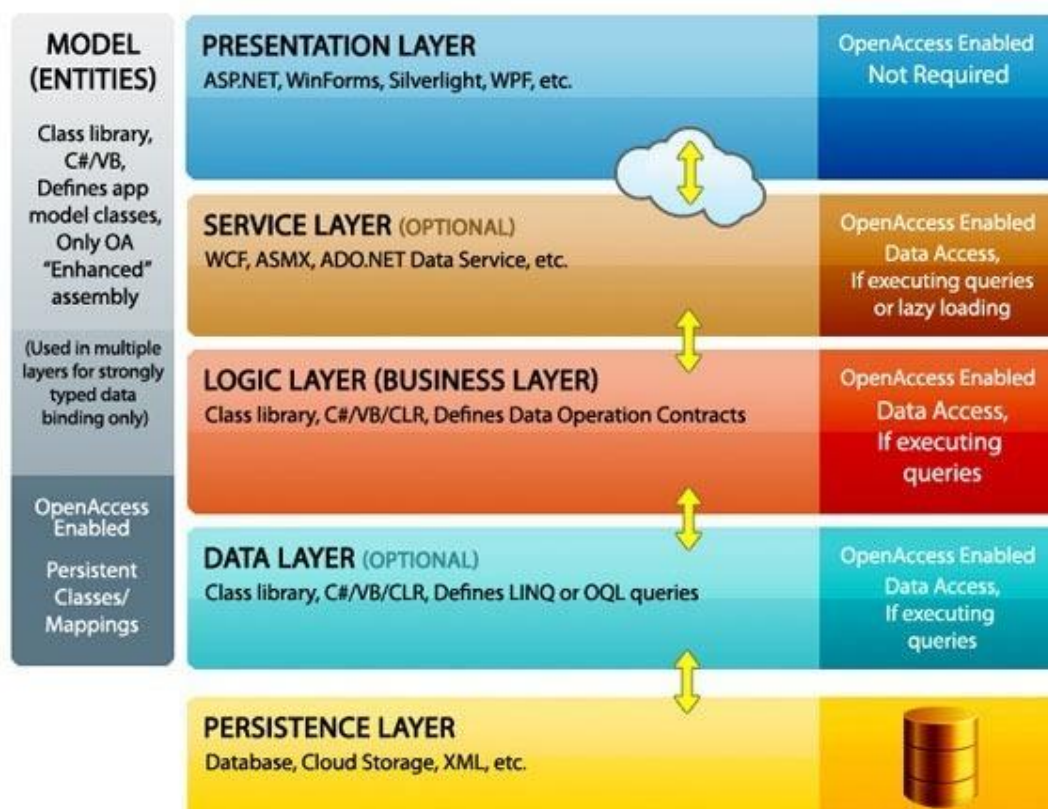
Tuy nhiên, khi quy mô tăng lên, Monolithic bộc lộ những vấn đề nghiêm trọng: mọi thay đổi nhỏ đều phải deploy lại toàn bộ ứng dụng; các module bị gắn kết chặt chẽ khiến một bug ở module A có thể làm sập toàn bộ hệ thống; việc mở rộng phải scale toàn bộ application thay vì chỉ scale component cần thiết.



1.1.2 Layered (N-Tier) Architecture

Kiến trúc phân tầng ra đời để giải quyết vấn đề tổ chức code trong Monolithic. Ứng dụng được chia thành các tầng logic theo chức năng: Presentation → Business Logic → Data Access → Database. Mỗi tầng chỉ giao tiếp với tầng liền kề, tạo ra sự phân tách trách nhiệm (Separation of Concerns).

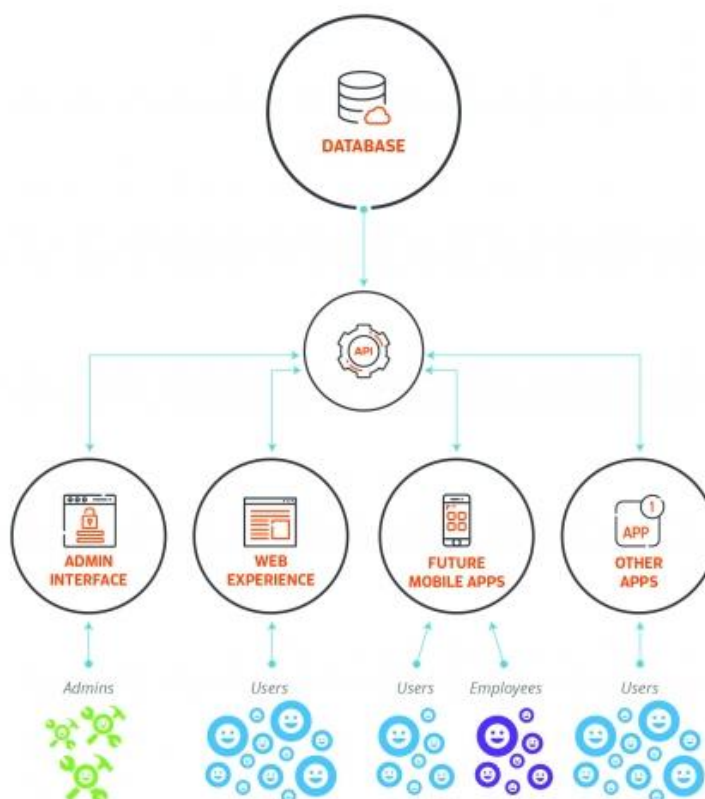
Tuy nhiên, phổ biến trong các dự án thực tế là "Layered architecture anti-pattern": tầng Data Access chứa đầy business logic, tầng Presentation gọi thẳng vào database. Sự phụ thuộc này chảy từ trên xuống nhưng cũng từ trung tâm ra ngoài, khiến toàn bộ hệ thống phụ thuộc vào chi tiết triển khai (implementation details).



1.1.3 API-Driven Architecture

Sự bùng nổ của thiết bị di động và Single Page Applications (SPA) thúc đẩy mô hình API-Driven: backend chỉ cung cấp RESTful API hoặc GraphQL, frontend (web, mobile, desktop) được phát triển độc lập và giao tiếp với backend qua API. Mô hình này giải phóng frontend và backend khỏi sự ràng buộc về công nghệ và tốc độ phát triển.

Trong ngữ cảnh của giáo trình này, .NET 10 Minimal APIs đóng vai trò backend API server, còn Next.js App Router đóng vai trò frontend client. Hai thành phần này giao tiếp hoàn toàn qua HTTP/JSON API — không có server-side rendering truyền thống, không có shared view engine.



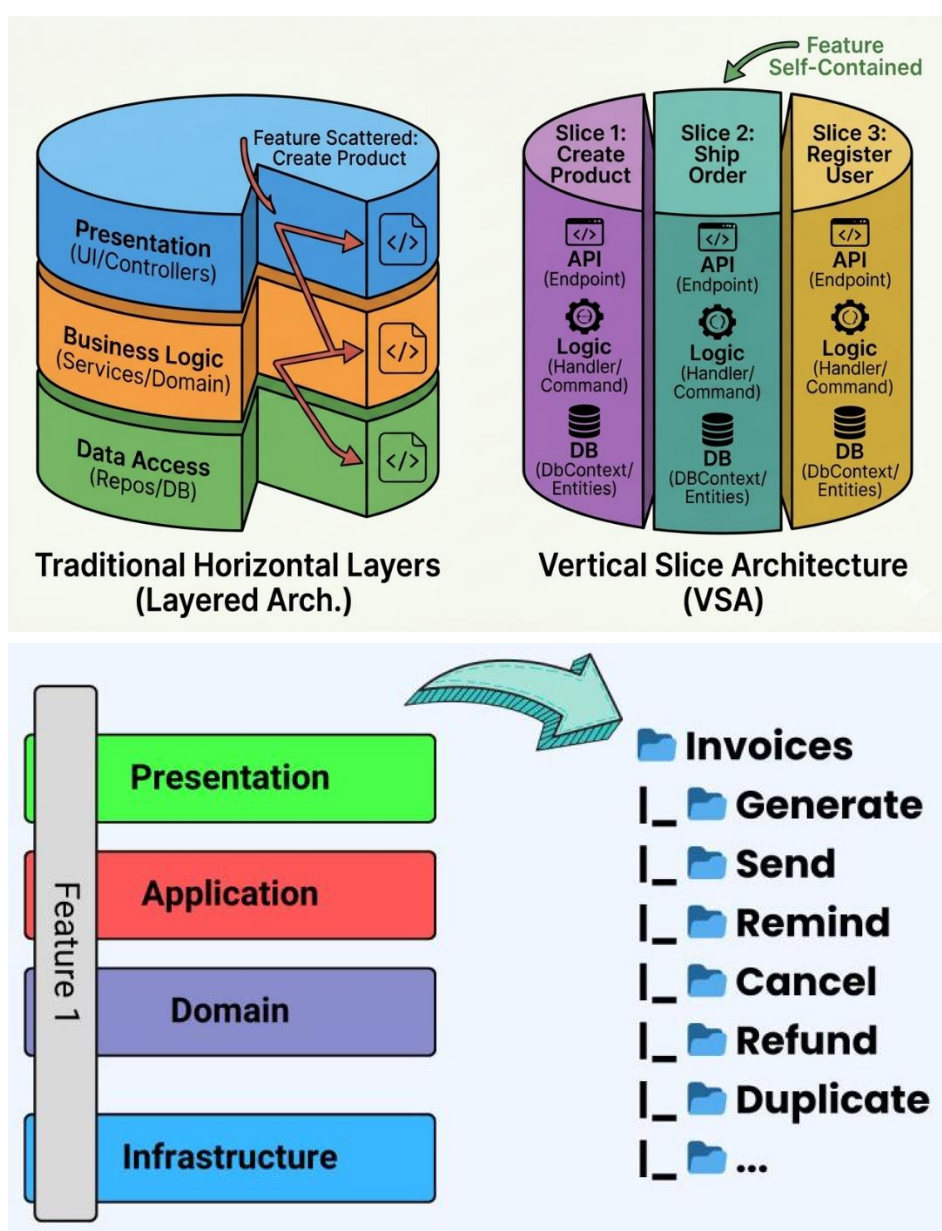
1.1.4 Vertical Slice Architecture

Vertical Slice Architecture (VSA) là cách tổ chức code theo tính năng (feature) thay vì theo tầng kỹ thuật. Thay vì folder /Controllers, /Services, /Repositories, VSA tổ chức code thành /Features/CreateRecipe/, /Features/GetRecipes/... Mỗi "slice" chứa tất cả code cần thiết cho một use case: request model, handler, response model, validation.

VSA kết hợp tự nhiên với CQRS/MediatR và là cách tổ chức phổ biến nhất trong Clean Architecture hiện đại. Khi một tính năng cần thay đổi, developer chỉ cần mở đúng một folder thay vì phải nhảy qua lại giữa nhiều tầng.

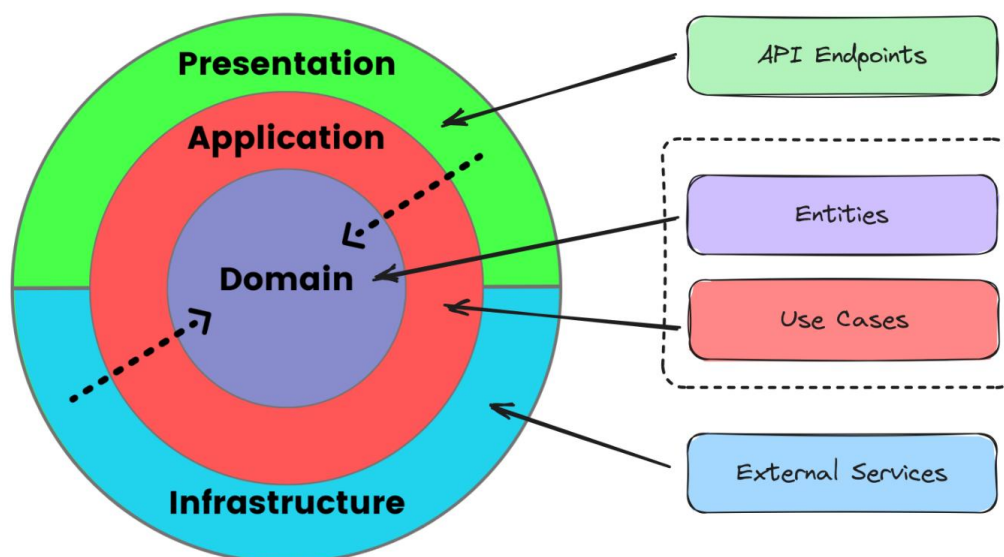
Tiêu chí	Monolithic	Layered	API-Driven + VSA
Tổ chức code	Theo module/feature	Theo tầng kỹ thuật	Theo feature (slice)
Coupling	Cao	Trung bình	Thấp
Khả năng scale	Khó (toàn bộ)	Trung bình	Cao (theo service)
Thời gian setup	Nhanh	Trung bình	Lâu hơn
Khả năng test	Khó	Trung bình	Dễ (isolation tốt)
Phù hợp với	Prototype, MVP nhỏ	Dự án trung bình	Dự án lớn, team nhiều người

Bảng 1.1. So sánh các mô hình kiến trúc ứng dụng web



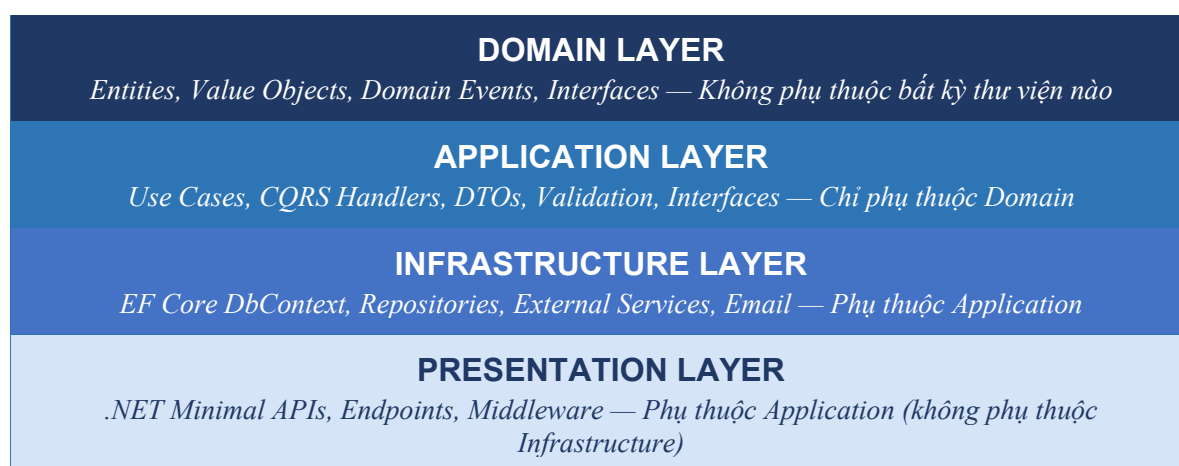
1.2. CLEAN ARCHITECTURE TRONG HỆ SINH THÁI .NET

Clean Architecture, được đề xuất bởi Robert C. Martin (Uncle Bob), là một mô hình kiến trúc dựa trên nguyên tắc cốt lõi: business rules (Domain và Application) không được phụ thuộc vào bất kỳ framework, database hay UI nào. Sự phụ thuộc luôn chảy từ ngoài vào trong — các tầng bên ngoài biết về tầng bên trong, nhưng chiều ngược lại thì không.



1.2.1 Các tầng trong Clean Architecture

Clean Architecture chia ứng dụng thành 4 tầng đồng tâm, với quy tắc phụ thuộc (Dependency Rule): source code dependencies chỉ được trỏ vào (inward). Tầng trong cùng không biết gì về tầng ngoài.



Hình 1.1. Sơ đồ các tầng Clean Architecture (phụ thuộc từ ngoài vào trong)

Nguyên tắc quan trọng nhất: Presentation Layer KHÔNG phụ thuộc Infrastructure Layer. Cả hai đều phụ thuộc Application Layer thông qua các interface (abstraction). Đây là điểm mấu chốt để đạt được khả năng kiểm thử (testability) — bạn có thể mock toàn bộ Infrastructure trong unit test mà không cần database thật.

1.2.2 Cấu trúc Solution cho Culinary Blog

Dưới đây là cấu trúc folder chuẩn cho dự án theo Clean Architecture. Mỗi tầng là một C# project riêng, với project dependencies phản ánh đúng quy tắc phụ thuộc theo chiều từ ngoài vào trong:



1.2.3 Dependency Inversion Principle và Dependency Injection

Dependency Inversion Principle (DIP) là nguyên tắc cuối cùng trong bộ SOLID: các module cấp cao không nên phụ thuộc vào module cấp thấp; cả hai nên phụ thuộc vào abstraction (interface). Trong .NET, cơ chế Dependency Injection (DI) tích hợp sẵn trong framework hiện thực hóa nguyên tắc này.

```
C# CulinaryBlog.Application/Contracts/Persistence/IApplicationDbContext.cs

// Abstraction – Application Layer định nghĩa "hợp đồng" với database
// Infrastructure Layer sẽ implement interface này
// Nhờ đó, Application không phụ thuộc trực tiếp vào EF Core
namespace CulinaryBlog.Application.Contracts.Persistence;

public interface IApplicationDbContext
{
    DbSet<Category> Categories { get; }
    DbSet<Recipe> Recipes { get; }

    // Lưu thay đổi – trả về số bản ghi bị ảnh hưởng
    Task<int> SaveChangesAsync(CancellationToken cancellationToken = default);
}
```

```
C# CulinaryBlog.Application/DependencyInjection.cs

// Extension method tập trung đăng ký tất cả service của Application Layer
// Program.cs chỉ cần gọi builder.Services.AddApplication()
using System.Reflection;
using Microsoft.Extensions.DependencyInjection;
using Mapster;
using MapsterMapper;

namespace CulinaryBlog.Application;

public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection
services)
    {
        // Đăng ký MediatR – tự động tìm tất cả Handler, Validator
        // trong Assembly của Application Layer
        services.AddMediatR(cfg =>
            cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly()));

        // Cấu hình Mapster – tự động đăng ký TypeAdapterConfig
        var config = TypeAdapterConfig.GlobalSettings;
        config.Scan(Assembly.GetExecutingAssembly());
        services.AddSingleton(config);
        services.AddScoped<IMapper, ServiceMapper>();

        return services;
    }
}
```

1.2.4 CQRS Pattern và MediatR

CQRS (Command Query Responsibility Segregation) là pattern phân tách hoạt động "đọc" (Query) và "ghi" (Command) thành hai luồng xử lý riêng biệt. Thay vì một Service class vừa đọc vừa ghi, mỗi use case được đại diện bởi một Command (hoặc Query) riêng với một Handler riêng — đây chính là biểu hiện của Vertical Slice.

Khái niệm

Command: Yêu cầu thay đổi trạng thái hệ thống (Create, Update, Delete). Command không trả về dữ liệu phức tạp, chỉ trả về kết quả (DTO mới tạo hoặc xác nhận thành công). Query: Yêu cầu lấy dữ liệu mà không thay đổi trạng thái hệ thống. Query có thể được tối ưu riêng (read model, caching) mà không ảnh hưởng write path.

MediatR là thư viện .NET implement Mediator pattern: thay vì component A gọi trực tiếp component B, cả hai đều giao tiếp qua một "trung gian" (Mediator). Trong CQRS, `ISender.Send(command)` sẽ tự động tìm và gọi `CommandHandler` tương ứng — Presentation Layer không cần biết Handler nào đang xử lý, không cần inject Service.

```
C# CulinaryBlog.Application/Features/Categories/Commands/  
CreateCategory/CreateCategoryCommand.cs
```

```
// Command – bất biến (immutable) vì nó đại diện cho một ý định đã xác định  
// record là lựa chọn hoàn hảo cho Command và Query trong C#  
using MediatR;  
using CulinaryBlog.Application.DTOs;  
  
namespace CulinaryBlog.Application.Features.Categories.Commands.CreateCategory;  
  
// IRequest<CategoryDto> nghĩa là Command này trả về CategoryDto sau khi xử lý  
public record CreateCategoryCommand(  
    string Name,  
    string? Description  
) : IRequest<CategoryDto>;
```

```
C# CulinaryBlog.Application/Features/Categories/Commands/  
CreateCategory/CreateCategoryCommandHandler.cs
```

```
using CulinaryBlog.Application.Contracts.Persistence;  
using CulinaryBlog.Application.DTOs;  
using CulinaryBlog.Domain.Entities;  
using Mapster;  
using MediatR;  
  
namespace CulinaryBlog.Application.Features.Categories.Commands.CreateCategory;  
  
public class CreateCategoryCommandHandler  
    : IRequestHandler<CreateCategoryCommand, CategoryDto>  
{  
    private readonly IApplicationDbContext _context;  
  
    // Constructor Injection – DI container tự động cung cấp  
    IApplicationDbContext  
    public CreateCategoryCommandHandler(IApplicationDbContext context)
```

```

{
    _context = context;
}

public async Task<CategoryDto> Handle(
    CreateCategoryCommand request,
    CancellationToken cancellationToken)
{
    // 1. Tạo entity qua Domain factory method – đảm bảo business rules
    var category = Category.Create(request.Name, request.Description);

    // 2. Thêm vào DbContext (chưa ghi vào database)
    _context.Categories.Add(category);

    // 3. Ghi vào database (thực thi SQL INSERT)
    await _context.SaveChangesAsync(cancellationToken);

    // 4. Map sang DTO để trả về – Presentation Layer không nhận raw Entity
    return category.Adapt<CategoryDto>();
}
}

```

1.2.5 Domain Entity và DTO Mapping với Mapper

Trong Clean Architecture, Domain Entity chứa business logic và không nên được tiết lộ trực tiếp ra ngoài. DTO (Data Transfer Object) là một class đơn giản chỉ chứa data, được dùng để giao tiếp giữa các tầng. Mapper là thư viện mapping hiệu năng cao thay thế AutoMapper, với khả năng compile-time type checking.

C# CulinaryBlog.Domain/Entities/Category.cs

```

namespace CulinaryBlog.Domain.Entities;

// Entity theo Domain-Driven Design:
// - Private setters: đảm bảo trạng thái chỉ được thay đổi qua business methods
// - Factory method: điểm khởi tạo duy nhất, đảm bảo entity luôn hợp lệ
public class Category
{
    public Guid Id { get; private set; }
    public string Name { get; private set; } = string.Empty;
    public string Slug { get; private set; } = string.Empty;
    public string? Description { get; private set; }
    public DateTime CreatedAt { get; private set; }
    public DateTime? UpdatedAt { get; private set; }

    // EF Core cần constructor không tham số – protected để hạn chế truy cập bên ngoài
    protected Category() { }

    // Factory method – đây là cách DUY NHẤT để tạo Category hợp lệ
    public static Category Create(string name, string? description = null)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(name, nameof(name));
    }
}

```

```

        return new Category
        {
            Id          = Guid.NewGuid(),
            Name        = name.Trim(),
            Slug        = SlugHelper.Generate(name), // Tạo slug: "Món Chính" →
"mon-chinh"
            Description = description?.Trim(),
            CreatedAt   = DateTime.UtcNow,
        };
    }

    // Business method – đóng gói logic update và cập nhật UpdatedAt tự động
    public void Update(string name, string? description)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(name, nameof(name));
        Name          = name.Trim();
        Slug          = SlugHelper.Generate(name);
        Description    = description?.Trim();
        UpdatedAt     = DateTime.UtcNow;
    }
}

```

C# CulinaryBlog.Application/DTOs/CategoryDto.cs

```

namespace CulinaryBlog.Application.DTOs;

// DTO – chỉ chứa data, không có logic
// Mapster sẽ tự động map từ Category Entity sang CategoryDto
// nhờ tên property trùng khớp (convention-based mapping)
public record CategoryDto(
    Guid Id,
    string Name,
    string Slug,
    string? Description,
    DateTime CreatedAt
);

```

1.3. THIẾT KẾ RESTFUL API

REST (Representational State Transfer) không phải là một giao thức hay đặc tả kỹ thuật — đó là một tập hợp các ràng buộc kiến trúc (architectural constraints) do Roy Fielding định nghĩa trong luận án tiến sĩ năm 2000. Một API được gọi là "RESTful" khi nó tuân thủ đủ 6 ràng buộc: Client-Server, Stateless, Cacheable, Uniform Interface, Layered System và Code on Demand (tùy chọn).

Trong thực tế phát triển, phần lớn trọng tâm nằm ở Uniform Interface — cụ thể là cách thiết kế URI, sử dụng HTTP methods và status codes một cách nhất quán và có ngữ nghĩa.

1.3.1 Resource-Oriented Design và URI Convention

Nguyên tắc cốt lõi của RESTful URI design: URI đại diện cho resource (danh từ), không phải action (động từ). HTTP method mới là thành phần biểu diễn action.

Sai ✗ (Action-based)	Đúng ✓ (Resource-based)	Giải thích
GET /getCategories	GET /categories	Collection resource — danh từ số nhiều
GET /getCategory?id=1	GET /categories/{id}	Single resource — ID trong path
POST /createCategory	POST /categories	Method POST đã thể hiện "create"
PUT /updateCategory/1	PUT /categories/{id}	Method PUT đã thể hiện "update"
DELETE /deleteCategory/1	DELETE /categories/{id}	Method DELETE đã thể hiện "delete"
GET /getRecipesByCategory/1	GET /categories/{id}/recipes	Nested resource — quan hệ phân cấp

Bảng 1.2. So sánh URI design đúng và sai trong RESTful API

1.3.2 HTTP Methods và ngữ nghĩa

Mỗi HTTP method mang một ngữ nghĩa cụ thể. Hai khái niệm quan trọng cần nắm: Safe (an toàn — không thay đổi state) và Idempotent (lũy đẳng — gọi nhiều lần cho cùng kết quả như gọi một lần).

Method	Ngữ nghĩa	Safe	Idempotent	Có Request Body	Status Code điển hình
GET	Lấy resource	✓	✓	✗	200 OK
POST	Tạo resource mới	✗	✗	✓	201 Created
PUT	Thay thế toàn bộ resource	✗	✓	✓	200 OK
PATCH	Cập nhật một phần	✗	✗	✓	200 OK
DELETE	Xoá resource	✗	✓	✗	204 No Content
HEAD	Như GET, không có body	✓	✓	✗	200 OK

Bảng 1.3. HTTP Methods và thuộc tính Safe/Idempotent

💡 Lưu ý

Idempotency trong PUT: Gọi PUT /recipes/1 với cùng một payload nhiều lần phải cho kết quả giống nhau. Điều này khác với POST: gọi POST /recipes nhiều lần sẽ tạo ra nhiều recipe khác nhau. Trong .NET, property [FromBody] kết hợp với HTTP Method trong Minimal API tự động đảm bảo điều này nếu bạn implement đúng logic.

1.3.3 HTTP Status Codes

Status code truyền đạt kết quả của request. Việc sử dụng đúng status code giúp client xử lý response đúng cách mà không cần parse body. Đây là phần thường bị implement sai trong các dự án thực tế.

Range	Ý nghĩa	Các code phổ biến
2xx	Thành công	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 304 Not Modified
4xx	Lỗi từ phía client	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx	Lỗi từ phía server	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Bảng 1.4. Các nhóm HTTP Status Code

Code	Tên	Khi nào dùng (ví dụ trong Culinary Blog)
200	OK	GET /recipes trả về danh sách; PUT /recipes/{id} cập nhật thành công
201	Created	POST /recipes tạo recipe mới thành công. Header Location chỉ đến resource mới
204	No Content	DELETE /recipes/{id} xóa thành công; PUT /categories/{id} (không cần trả về body)
400	Bad Request	Dữ liệu gửi lên sai format, thiếu field bắt buộc, giá trị không hợp lệ
401	Unauthorized	Chưa đăng nhập (thiếu JWT token hoặc token đã hết hạn)
403	Forbidden	Đã đăng nhập nhưng không có quyền — ví dụ sửa recipe của người khác
404	Not Found	Recipe với ID được yêu cầu không tồn tại trong database
409	Conflict	Tạo Category với tên đã tồn tại (duplicate slug)
422	Unprocessable Entity	Dữ liệu đúng format nhưng vi phạm business rule
429	Too Many Requests	Vượt quá rate limit — cần thiết cho endpoint Login

Bảng 1.5. Các HTTP Status Code thường dùng trong Culinary Blog API

1.3.4 API Versioning

API Versioning là kỹ thuật quản lý các phiên bản khác nhau của API, cho phép giới thiệu breaking changes mà không làm hỏng các client cũ. Có ba chiến lược phổ biến trong .NET: URL Path Versioning, Query String Versioning và Header Versioning.

Trong giáo trình này, chúng ta dùng URL Path Versioning vì nó rõ ràng nhất, dễ debug và tương thích tốt với Scalar documentation.

Text CulinaryBlog.API/Endpoints/CategoryEndpoints.cs (phần cấu hình group)

```
// URL Path Versioning: /api/v1/categories, /api/v2/categories
// Khi cần breaking change: tạo /api/v2 với interface mới,
// giữ nguyên /api/v1 để không làm hỏng client cũ

var v1Group = app.MapGroup("/api/v1/categories")
    .WithTags("Categories")
    .WithGroupName("v1")
    .WithOpenApi();

// Tất cả endpoints của version 1 đăng ký vào v1Group
v1Group.MapGet("/", HandleGetCategories);
v1Group.MapGet("/{id}", HandleGetCategoryById);
v1Group.MapPost("/", HandleCreateCategory);
v1Group.MapPut("/{id}", HandleUpdateCategory);
v1Group.MapDelete("/{id}", HandleDeleteCategory);
```

1.3.5 Pagination, Filtering và Sorting

Một API trả về toàn bộ dữ liệu mà không phân trang là không thể chấp nhận trong production. Culinary Blog có thể có hàng nghìn công thức nấu ăn — việc trả về tất cả trong một request không chỉ chậm mà còn có thể gây DoS cho server.

Có hai chiến lược phân trang chính: Offset-based và Cursor-based. Offset-based đơn giản hơn (dùng page number) nhưng có vấn đề với "missing rows" khi dữ liệu thay đổi. Cursor-based dùng một giá trị con trỏ (thường là ID hoặc createdAt) để đảm bảo tính nhất quán.

C# CulinaryBlog.Application/Common/Models/PaginatedResult.cs

```
namespace CulinaryBlog.Application.Common.Models;

/// <summary>
/// Generic wrapper cho mọi kết quả phân trang trong hệ thống.
/// Chứa đủ metadata để client tự xây dựng navigation links.
/// </summary>
/// <typeparam name="T">Kiểu dữ liệu của từng item trong danh sách</typeparam>
public sealed class PaginatedResult<T>
{
    // Danh sách items trong trang hiện tại
    public IReadOnlyList<T> Items { get; init; }
    // Tổng số bản ghi (không phân trang) – cần để tính TotalPages
    public int TotalCount { get; init; }
    // Trang hiện tại (bắt đầu từ 1)
    public int Page { get; init; }
    // Số item trên một trang
    public int PageSize { get; init; }
}
```



```

// Tính toán từ các giá trị trên – không lưu trong database
public int TotalPages => (int)Math.Ceiling(TotalCount /
(double)PageSize);
public bool HasNextPage => Page < TotalPages;
public bool HasPreviousPage => Page > 1;

public PaginatedResult(
    IReadOnlyList<T> items,
    int totalCount, int page, int pageSize)
{
    Items = items;
    TotalCount = totalCount;
    Page = page;
    PageSize = pageSize;
}
}

```

C#

CulinaryBlog.Application/Features/Categories/Queries/GetCategories/GetCategoriesQuery.cs

```

using CulinaryBlog.Application.Common.Models;
using CulinaryBlog.Application.DTOs;
using MediatR;

namespace CulinaryBlog.Application.Features.Categories.Queries.GetCategories;

// Query với đầy đủ tham số phân trang, lọc, sắp xếp
// Default values đảm bảo client không cần gửi tất cả tham số
public record GetCategoriesQuery(
    int Page = 1, // Trang hiện tại
    int PageSize = 10, // Số item/trang, max nên giới hạn 100
    string? Search = null, // Tìm kiếm theo tên/mô tả
    string SortBy = "name", // Tên field để sắp xếp
    bool Descending = false // Chiều sắp xếp
) : IRequest<PaginatedResult<CategoryDto>>;

```

C#

CulinaryBlog.Application/Features/Categories/Queries/GetCategories/GetCategoriesQueryHandler.cs

```

using CulinaryBlog.Application.Common.Models;
using CulinaryBlog.Application.Contracts.Persistence;
using CulinaryBlog.Application.DTOs;
using Mapster;
using MediatR;
using Microsoft.EntityFrameworkCore;

namespace CulinaryBlog.Application.Features.Categories.Queries.GetCategories;

public class GetCategoriesQueryHandler
    : IRequestHandler<GetCategoriesQuery, PaginatedResult<CategoryDto>>
{
    private readonly IApplicationDbContext _context;

    public GetCategoriesQueryHandler(IApplicationDbContext context)
        => _context = context;
}

```

```

public async Task<PaginatedResult<CategoryDto>> Handle(
    GetCategoriesQuery request,
    CancellationToken cancellationToken)
{
    // AsNoTracking(): không cần tracking vì chỉ đọc dữ liệu (Query, không phải
    Command)
    var query = _context.Categories.AsNoTracking();

    // Áp dụng bộ lọc tìm kiếm – chỉ thêm điều kiện WHERE khi có giá trị
    if (!string.IsNullOrEmpty(request.Search))
    {
        var search = request.Search.ToLower().Trim();
        query = query.Where(c =>
            c.Name.ToLower().Contains(search) ||
            (c.Description != null && c.Description.ToLower().Contains(search)));
    }

    // COUNT trước khi phân trang – đây là truy vấn SQL riêng biệt
    var totalCount = await query.CountAsync(cancellationToken);

    // Sắp xếp động dựa trên tham số request
    query = (request.SortBy.ToLower(), request.Descending) switch
    {
        ("name", false) => query.OrderBy(c => c.Name),
        ("name", true) => query.OrderByDescending(c => c.Name),
        ("createdat", false) => query.OrderBy(c => c.CreatedAt),
        ("createdat", true) => query.OrderByDescending(c => c.CreatedAt),
        _ => query.OrderBy(c => c.Name) // default
    };

    // Phân trang: Skip bỏ qua các trang trước, Take lấy đúng số item cần
    // SQL tương đương: OFFSET (page-1)*pageSize ROWS FETCH NEXT pageSize ROWS ONLY
    var items = await query
        .Skip((request.Page - 1) * request.PageSize)
        .Take(request.PageSize)
        .ProjectToType<CategoryDto>() // Mapper projection: chỉ SELECT cột cần
        thiết
        .ToListAsync(cancellationToken);

    return new PaginatedResult<CategoryDto>(
        items, totalCount, request.Page, request.PageSize);
}

```

☒ Mẹo

ProjectToType<T>() của Mapster (tương tự ProjectTo<T>() của AutoMapper) tạo ra SQL SELECT chỉ lấy các cột cần thiết thay vì SELECT *. Với Recipe entity có 15 fields nhưng RecipeDto chỉ cần 8, điều này giảm đáng kể lượng data truyền qua network và bộ nhớ cần thiết.

1.3.6 Chuẩn response JSON cho danh sách phân trang

Dưới đây là ví dụ response JSON chuẩn cho endpoint GET `/api/v1/categories?page=2&pageSize=5`:

Text Response: GET /api/v1/categories?page=2&pageSize=5&search=monkho

```
{
  "items":
    {
      "id": "a1b2c3d4-...",
      "name": "Món Kho",
      "slug": "mon-kho",
      "description": "Các món ăn kho truyền thống Việt Nam",
      "createdAt": "2025-01-15T08:30:00Z"
    }
  ],
  "totalCount": 3,
  "page": 2,
  "pageSize": 5,
  "totalPages": 1,
  "hasNextPage": false,
  "hasPreviousPage": true
}
```

1.4. OPENAPI SPECIFICATION VỚI SCALAR

OpenAPI Specification (trước đây là Swagger) là một tiêu chuẩn mô tả RESTful API dưới dạng file YAML hoặc JSON. OpenAPI cho phép tự động tạo tài liệu API, generate client SDK, và kiểm thử API mà không cần viết tài liệu thủ công.

Từ .NET 9, Microsoft đã tích hợp OpenAPI native vào framework và không còn đề xuất Swashbuckle. .NET 10 mặc định sử dụng Scalar làm API documentation UI — một công cụ hiện đại hơn, đẹp hơn và hỗ trợ nhiều tính năng hơn Swagger UI.

1.4.1 Cấu hình OpenAPI và Scalar trong .NET 10

C# CulinaryBlog.API/Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Đăng ký các service của các layer
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

// Đăng ký OpenAPI – .NET 10 native, không cần Swashbuckle
builder.Services.AddOpenApi(options =>
{
    options.AddDocumentTransformer((document, context, ct) =>
    {
        // Thông tin cơ bản của API
        document.Info = new()
        {
            Title = "Culinary Blog API",
```

```

        Version = "v1",
        Description = "API cho ứng dụng Blog ẩm thực và nấu ăn",
    };
    return Task.CompletedTask;
});
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    // Endpoint phục vụ OpenAPI spec (JSON/YAML)
    app.MapOpenApi();

    // Scalar UI – thay thế Swagger UI trong .NET 10
    app.MapScalarApiReference(options =>
    {
        options
            .WithTitle("Culinary Blog API")
            .WithTheme(ScalarTheme.Purple) // Giao diện tím phù hợp với food
            .WithDefaultHttpClient(ScalarTarget.CSharp,
            ScalarClient.HttpClient);
    });
}

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();

// Đăng ký tất cả endpoints
app.MapCategoryEndpoints();
app.MapRecipeEndpoints();

app.Run();

```

1.4.2 Tài liệu hóa Endpoints với OpenAPI metadata

```

C# CulinaryBlog.API/Endpoints/CategoryEndpoints.cs

public static IEndpointRouteBuilder MapCategoryEndpoints(
    this IEndpointRouteBuilder app)
{
    var group = app.MapGroup("/api/v1/categories")
        .WithTags("Categories") // Nhóm trong Scalar UI
        .WithOpenApi();

    // GET – Lấy danh sách phân trang
    group.MapGet("/", async (
        [AsParameters] GetCategoriesQuery query, // Bind query string tự động
        ISender sender,
        CancellationToken ct) =>
    {
        var result = await sender.Send(query, ct);
        return Results.Ok(result);
    })
    .WithName("GetCategories")

```

```

.WithSummary("Lấy danh sách danh mục có phân trang")
.WithDescription("Hỗ trợ search, sort, và pagination. " +
    "Sử dụng ?search=... để tìm kiếm theo tên hoặc mô tả.")
.Produces<PaginatedResult<CategoryDto>>(200);

// POST – tạo danh mục mới
group.MapPost("/", async (
    CreateCategoryCommand command,
    ISender sender,
    CancellationToken ct) =>
{
    var result = await sender.Send(command, ct);
    // Trả về 201 Created với Location header trỏ đến resource mới
    return Results.Created($" /api/v1/categories/{result.Id}", result);
})
.WithName("CreateCategory")
.WithSummary("Tạo danh mục mới")
.Produces<CategoryDto>(201)
.ProducesProblem(400) // Validation error
.ProducesProblem(409); // Duplicate slug

return app;
}

```

1.5. HIỆN THỰC HÓA VỚI .NET 10 MINIMAL APIS VÀ POSTGRESQL

.NET 10 Minimal APIs là cách tiếp cận xây dựng HTTP API tối giản, không cần Controller class, ít boilerplate hơn so với ASP.NET MVC truyền thống. Được giới thiệu từ .NET 6 và ngày càng hoàn thiện, Minimal APIs ngày nay hỗ trợ đầy đủ tất cả tính năng cần thiết cho production: route groups, filters, dependency injection, middleware, OpenAPI.

1.5.1 Infrastructure Layer — DbContext và DI

```

C# CulinaryBlog.Infrastructure/Persistence/ApplicationDbContext.cs

using CulinaryBlog.Application.Contracts.Persistence;
using CulinaryBlog.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace CulinaryBlog.Infrastructure.Persistence;

// ApplicationDbContext implement IApplicationDbContext (interface từ
// Application Layer)
// → Application Layer không phụ thuộc EF Core, chỉ phụ thuộc interface
public class ApplicationDbContext : DbContext, IApplicationDbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Recipe> Recipes => Set<Recipe>();
}

```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Áp dụng tất cả Entity Configuration trong assembly này
    // Thay vì cấu hình từng entity ở đây, ta tách ra file riêng
    modelBuilder.ApplyConfigurationsFromAssembly(
        typeof(ApplicationDbContext).Assembly);

    base.OnModelCreating(modelBuilder);
}
}
```

C# CulinaryBlog.Infrastructure/DependencyInjection.cs

```
using CulinaryBlog.Application.Contracts.Persistence;
using CulinaryBlog.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace CulinaryBlog.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        // Đăng ký PostgreSQL với EF Core
        services.AddDbContext<ApplicationDbContext>(options =>
        {
            options.UseNpgsql(
                configuration.GetConnectionString("DefaultConnection"),
                npgsql => npgsql.MigrationsAssembly(
                    typeof(ApplicationDbContext).Assembly.FullName));
        });

        // Đăng ký IApplicationDbContext → ApplicationDbContext
        // Scoped: mỗi HTTP request có một DbContext instance riêng
        services.AddScoped<IApplicationDbContext, ApplicationDbContext>();

        return services;
    }
}
```

JSON appsettings.Development.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
    "Host=localhost;Port=5432;Database=culinary_blog;Username=postgres;Password=yourpassword"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",

```



```
"Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  }
}
```

☑ Mẹo

Với .NET 10 và Aspire (hoặc Docker Compose), bạn có thể khởi chạy PostgreSQL tự động trong development mà không cần cài đặt thủ công. Chương 6 sẽ hướng dẫn chi tiết cách container hóa toàn bộ stack. Hiện tại, cài đặt PostgreSQL local (port 5432, user postgres) là đủ.

1.6. CASE STUDY: KHỞI TẠO NỀN TẢNG CULINARY BLOG

Trong Case Study này, chúng ta sẽ đi qua toàn bộ quá trình khởi tạo dự án Culinary Blog từ đầu: tạo solution structure, cài đặt NuGet packages, xây dựng Domain Entity, Application Layer với CQRS, Infrastructure Layer với EF Core, và Presentation Layer với Minimal APIs.

Bước 1: Tạo Solution và Các Projects

Text Terminal – Tạo Clean Architecture Solution

```
# Tạo solution
dotnet new sln -n CulinaryBlog

# Tạo 4 projects tương ứng 4 tầng
dotnet new classlib -n CulinaryBlog.Domain -o src/CulinaryBlog.Domain
dotnet new classlib -n CulinaryBlog.Application -o
src/CulinaryBlog.Application
dotnet new classlib -n CulinaryBlog.Infrastructure -o
src/CulinaryBlog.Infrastructure
dotnet new webapi -n CulinaryBlog.API -o src/CulinaryBlog.API

# Thêm tất cả vào solution
dotnet sln add src/CulinaryBlog.Domain/CulinaryBlog.Domain.csproj
dotnet sln add src/CulinaryBlog.Application/CulinaryBlog.Application.csproj
dotnet sln add
src/CulinaryBlog.Infrastructure/CulinaryBlog.Infrastructure.csproj
dotnet sln add src/CulinaryBlog.API/CulinaryBlog.API.csproj

# Thiết lập Project References (thể hiện Dependency Rule)
dotnet add src/CulinaryBlog.Application/CulinaryBlog.Application.csproj
reference \
  src/CulinaryBlog.Domain/CulinaryBlog.Domain.csproj

dotnet add src/CulinaryBlog.Infrastructure/CulinaryBlog.Infrastructure.csproj
reference \
  src/CulinaryBlog.Application/CulinaryBlog.Application.csproj

dotnet add src/CulinaryBlog.API/CulinaryBlog.API.csproj reference \
  src/CulinaryBlog.Application/CulinaryBlog.Application.csproj \
```

```
src/CulinaryBlog.Infrastructure/CulinaryBlog.Infrastructure.csproj
```

Bước 2: Cài đặt NuGet Packages

Text Terminal – Cài đặt NuGet Packages

```
# Application Layer – MediatR và Mapster
dotnet add src/CulinaryBlog.Application package MediatR
dotnet add src/CulinaryBlog.Application package Mapster
dotnet add src/CulinaryBlog.Application package Mapster.DependencyInjection

# Infrastructure Layer – EF Core với PostgreSQL
dotnet add src/CulinaryBlog.Infrastructure package Microsoft.EntityFrameworkCore
dotnet add src/CulinaryBlog.Infrastructure package
Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add src/CulinaryBlog.Infrastructure package
Microsoft.EntityFrameworkCore.Tools

# API Layer – Scalar cho .NET 10
dotnet add src/CulinaryBlog.API package Scalar.AspNetCore
```

Bước 3: Domain Layer — Entity Category

Sau khi tạo projects, chúng ta bắt đầu từ tầng trong cùng — Domain Layer. Đây là code KHÔNG phụ thuộc vào bất kỳ NuGet package nào ngoài .NET runtime.

C# src/CulinaryBlog.Domain/Entities/Category.cs

```
namespace CulinaryBlog.Domain.Entities;

public class Category
{
    public Guid Id { get; private set; }
    public string Name { get; private set; } = string.Empty;
    public string Slug { get; private set; } = string.Empty;
    public string? Description { get; private set; }
    public DateTime CreatedAt { get; private set; }
    public DateTime? UpdatedAt { get; private set; }

    protected Category() { }

    public static Category Create(string name, string? description = null)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(name, nameof(name));
        return new Category
        {
            Id = Guid.NewGuid(),
            Name = name.Trim(),
            Slug = GenerateSlug(name),
            Description = description?.Trim(),
            CreatedAt = DateTime.UtcNow,
        };
    }
}
```

```

public void Update(string name, string? description)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(name, nameof(name));
    Name      = name.Trim();
    Slug      = GenerateSlug(name);
    Description = description?.Trim();
    UpdatedAt  = DateTime.UtcNow;
}

// Tạo URL slug từ tên: "Món Tráng Miếng" → "mon-trang-mieng"
private static string GenerateSlug(string name)
{
    return name.ToLowerInvariant()
        .Normalize(System.Text.NormalizationForm.FormD)
        .Replace(" ", "-")
        .Replace("đ", "d")
        .Replace("Đ", "d");
    // Lưu ý: Production nên dùng package Slugify.Core để xử lý Unicode đầy
    // đủ
}

```

Bước 4: Application Layer — CRUD Commands và Queries

Chúng ta sẽ implement đầy đủ CRUD cho Category: CreateCategoryCommand, UpdateCategoryCommand, DeleteCategoryCommand, GetCategoriesQuery và GetCategoryByIdQuery. Dưới đây là DeleteCategoryCommand làm ví dụ bổ sung:

```

C#
src/CulinaryBlog.Application/Features/Categories/Commands/DeleteCategory/DeleteCategoryCommand.cs

using MediatR;

namespace CulinaryBlog.Application.Features.Categories.Commands.DeleteCategory;

// Command xóa category – chỉ cần ID, không trả về data (Unit = void trong MediatR)
public record DeleteCategoryCommand(Guid Id) : IRequest<Unit>;

// — Handler —
public class DeleteCategoryCommandHandler : IRequestHandler<DeleteCategoryCommand, Unit>
{
    private readonly IApplicationDbContext _context;

    public DeleteCategoryCommandHandler(IApplicationDbContext context)
        => _context = context;

    public async Task<Unit> Handle(
        DeleteCategoryCommand request,
        CancellationToken cancellationToken)
    {
        // Tìm entity – ném NotFoundException nếu không tồn tại
        // (NotFoundException sẽ được catch ở Global Exception Handler – Chương 5)
        var category = await _context.Categories
            .FindAsync([request.Id], cancellationToken)
            ?? throw new NotFoundException(nameof(Category), request.Id);
    }
}

```

```
        _context.Categories.Remove(category);  
        await _context.SaveChangesAsync(cancellationToken);  
  
        return Unit.Value; // Tương đương void trong MediatR  
    }  
}
```

Bước 5: Kiểm tra bằng Scalar

Sau khi chạy ứng dụng với lệnh `dotnet run` trong thư mục `CulinaryBlog.API`, truy cập <https://localhost:5001/scalar/v1> để mở Scalar UI. Tại đây bạn có thể:

- Xem tất cả endpoints được document tự động từ OpenAPI metadata.
- Thực hiện request trực tiếp từ browser — tương tự Postman nhưng tích hợp sẵn.
- Kiểm tra response format, status codes và headers.
- Export OpenAPI spec (JSON/YAML) để dùng với các công cụ khác.

☑ Mẹo

Shortcut workflow: (1) Chạy `dotnet watch run` để hot-reload khi code thay đổi; (2) Mở Scalar tại `/scalar/v1`; (3) Dùng "Try it" để gửi request POST `/categories` với body `{"name": "Món Chính"}`; (4) Kiểm tra response 201 Created với Location header và body là `CategoryDto`.

CÂU HỎI THẢO LUẬN

1. Phân tích ưu và nhược điểm của Monolithic Architecture. Trong trường hợp nào bạn vẫn nên chọn Monolithic cho một dự án mới trong năm 2025?
2. Giải thích Dependency Rule trong Clean Architecture. Tại sao Application Layer không được import trực tiếp từ Infrastructure Layer? Điều gì xảy ra nếu vi phạm quy tắc này?
3. So sánh CQRS pattern với Service pattern truyền thống (một `UserService` với các method `CreateUser`, `GetUser`, `UpdateUser`...). Khi nào CQRS là lựa chọn tốt hơn và khi nào thì over-engineering?
4. Trong RESTful API design, khi nào nên dùng PUT và khi nào nên dùng PATCH? Hãy đưa ra ví dụ cụ thể trong context của Culinary Blog.
5. Cursor-based pagination có ưu điểm gì so với Offset-based pagination? Trường hợp sử dụng nào bắt buộc phải dùng cursor-based (ví dụ: real-time feed)?
6. Tại sao Scalar được ưa chuộng hơn Swagger UI trong .NET 10? Liệt kê ít nhất 3 điểm khác biệt cụ thể.
7. Giải thích tại sao DTO (Data Transfer Object) quan trọng trong Clean Architecture. Điều gì xảy ra nếu bạn trả về Entity trực tiếp từ API endpoint?

- Trong Context của Culinary Blog, hãy thiết kế URI scheme đầy đủ cho các resources: Recipe, Category, Comment, User, RecipeImage. Giải thích lý do cho từng lựa chọn.

BÀI TẬP THỰC HÀNH

Bài 1 — Thiết lập môi trường và khởi tạo project (Cơ bản)

- Cài đặt .NET 10 SDK, PostgreSQL và pgAdmin trên máy local.
- Tạo solution CulinaryBlog theo đúng Clean Architecture structure như hướng dẫn trong Case Study.
- Cài đặt tất cả NuGet packages cần thiết.
- Implement đầy đủ Category entity với hai method Create() và Update().
- Chạy ứng dụng và verify Scalar UI hoạt động tại /scalar/v1.

Bài 2 — Mở rộng Domain và Application Layer (Trung bình)

- Tạo Recipe entity với các properties: Id, Title, Description, Instructions, PrepTimeMinutes, CookTimeMinutes, Servings, Difficulty (enum: Easy/Medium/Hard), CategoryId, AuthorId, CreatedAt, UpdatedAt.
- Implement CreateRecipeCommand và GetRecipesQuery với đầy đủ filtering (theo CategoryId, Difficulty), sorting và pagination.
- Implement GetRecipeByIdQuery trả về RecipeDetailDto bao gồm cả thông tin Category.
- Cấu hình IEntityConfiguration<Recipe> trong Infrastructure Layer.
- Tạo EF Core migration và apply vào database PostgreSQL.

Bài 3 — API Design và Documentation (Nâng cao)

- Thiết kế và implement full RESTful CRUD API cho cả Category và Recipe.
- Đảm bảo tất cả endpoints được document đầy đủ trong Scalar (summary, description, response types).
- Implement nested resource: GET /api/v1/categories/{id}/recipes — lấy tất cả recipes thuộc một category.
- Thêm custom filter cho endpoint GET /recipes: lọc theo khoảng thời gian nấu (minCookTime/maxCookTime), lọc theo mức độ khó.
- Viết curl commands hoặc HTTP file (.http) để test tất cả endpoints.

TÓM TẮT NỘI DUNG CỐT LÕI

Khái niệm / Pattern	Mục đích	Trong Culinary Blog
Clean Architecture	Phân tách concerns, business rules độc lập với framework	4 projects: Domain, Application, Infrastructure, API

Khái niệm / Pattern	Mục đích	Trong Culinary Blog
CQRS + MediatR	Tách read/write, Vertical Slice tổ chức theo feature	CreateCategoryCommand, GetCategoriesQuery...
Dependency Inversion	Application không phụ thuộc Infrastructure	IApplicationDbContext interface
DTO + Mapper	Không expose Entity ra ngoài, transform data	CategoryDto, RecipeDto
RESTful Design	URI = resource, Method = action, codes có ngữ nghĩa	/api/v1/categories, /api/v1/recipes/{id}
Pagination	Kiểm soát lượng data trả về, tránh quá tải server	PaginatedResult<T> với Page, PageSize, TotalCount
OpenAPI + Scalar	API documentation tự động, interactive testing	Truy cập /scalar/v1 trong development
.NET 10 Minimal APIs	HTTP API tối giản, ít boilerplate	MapCategoryEndpoints(), MapRecipeEndpoints()

Các từ khóa quan trọng cần ghi nhớ:

Clean Architecture, CQRS, [MediatR](#), Vertical Slice Architecture, [IRequest<T>](#), [IRequestHandler<T,R>](#), Dependency Inversion Principle, Dependency Injection, DTO, [Mapper](#), [ProjectToType<T>\(\)](#), REST constraints, Idempotency, Pagination, [PaginatedResult<T>](#), OpenAPI 3.x, [Scalar](#), [.NET 10 Minimal APIs](#), [AsNoTracking\(\)](#)

TÀI LIỆU THAM KHẢO MỞ RỘNG

Sách và tài liệu học thuật:

- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall. — Nguồn gốc lý thuyết Clean Architecture.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. — Repository Pattern, Unit of Work, Data Mapper.
- Richardson, C. (2018). Microservices Patterns. Manning. — CQRS và Event Sourcing chuyên sâu.

Tài liệu kỹ thuật chính thức:

- Microsoft Learn — ASP.NET Core Minimal APIs: <https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis>
- Microsoft Learn — .NET 10 OpenAPI: <https://learn.microsoft.com/aspnet/core/fundamentals/openapi>
- Scalar Documentation: <https://scalar.com/docs>
- MediatR Wiki: <https://github.com/jbogard/MediatR/wiki>
- Mapper Documentation: <https://github.com/MapsterMapper/Mapster/wiki>

Bài viết và video:

- Milan Jovanović — Clean Architecture with .NET series:
<https://www.milanjovanovic.tech/blog/clean-architecture-folder-structure>
- Nick Chapsas — MediatR và CQRS trong .NET: YouTube channel "Nick Chapsas"
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Luận án tiến sĩ) — Định nghĩa gốc của REST:
<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Hết Chương 1